

08. Project Overview

保守担当者向け / Version 1.0 / シーシャバー「ひふみ」店舗管理サブシステム

本書は、このシステムを引き継ぐ・保守する担当者が最初に読む1冊である。全体像と「触るときに気をつけること」に絞って記載し、詳細は各設計書へ委ねる。まず1章と7章(設計上の不変条件)を読めば、事故を起こさずに変更に着手できる。

1. システム全体像

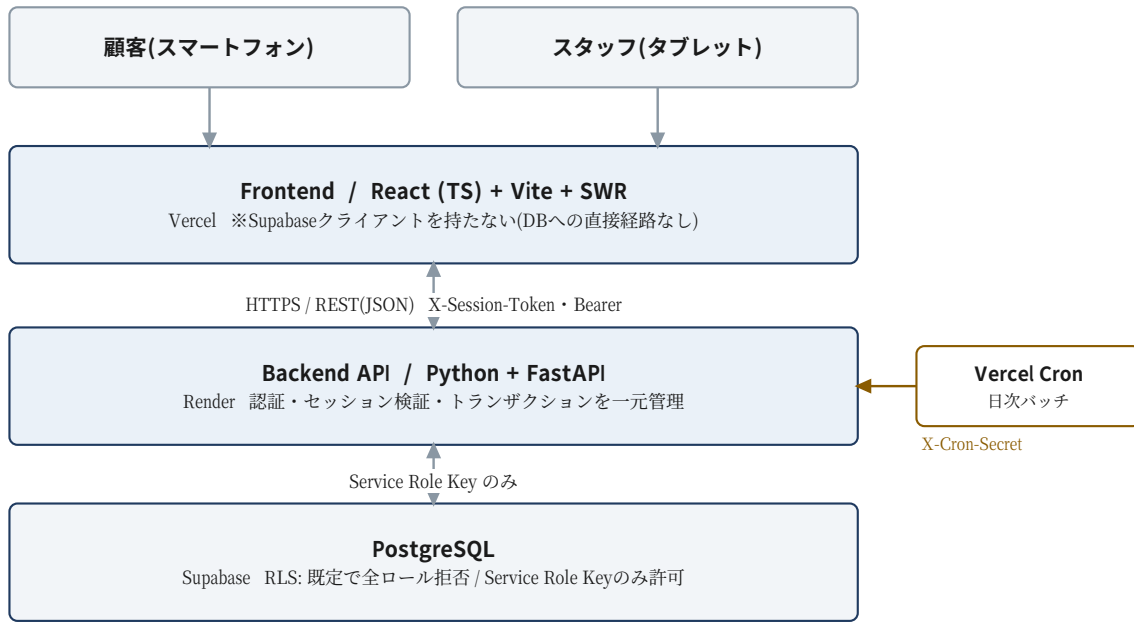


図1 システム構成図

レイヤー	技術	ホスティング	責務
Frontend	React (TS) + Vite + SWR	Vercel	画面表示とAPI呼び出しのみ。DBへの直接経路を持たない。
Backend API	Python + FastAPI	Render	認証・セッション検証・業務ロジック・トランザクション。全ての読み書きの唯一の窓口。
Database	PostgreSQL	Supabase	データ永続化。RLSで既定拒否、Service Role Keyのみ通過。
Batch	Vercel Cron	Vercel	日次で閉め忘れ卓を強制クリア。

1.1 外部サービスと停止時の影響

サービス	用途	停止したときに起きること
Vercel	フロントエンド配信 / Cron	画面が開けない。営業はレジで継続可能。
Render	API	注文・提供操作が一切できない。営業はレジで継続可能。
Supabase	DB	同上。復旧後、停止中の注文は失われる。

障害時に営業は止まらない

このシステムは決済を扱わないため、全面停止してもレジでの営業は継続できる。復旧を急ぐ必要はあるが、緊急度は「営業停止」ではなく「オペレーション効率の一時低下」と評価してよい。スタッフ側の運用は [07_利用ガイド] 6.3節に記載済み。

2. セキュリティ・認証の概要

このシステムの設計上、最も重要なのは「他卓へのなりすまし防止」と「DBへの直接アクセス遮断」の2点である。この2つが崩れると顧客の注文情報が漏洩するため、変更時は必ず7章の不変条件を確認すること。

2.1 3種類の認証

対象	方式	ヘッダー	発行タイミング	失効
スタッフ	JWT	Authorization: Bearer	ログイン時	8時間で自動失効
顧客	session_token (UUID)	X-Session-Token	卓の開栓時	会計クリア / 再開栓時
パッチ	共有シークレット	X-Cron-Secret	環境変数に固定	手動ローテーション

2.2 なりすまし防止の考え方

- QRコードには卓のID(連番)しか埋め込まれていないため、卓IDだけでは認証にならない。
- 開栓のたびに発行される session_token を、顧客系の全APIで毎回照合する。不一致・未指定・会計済セッションはいずれも 403 (SESSION_EXPIRED)。
- この検証はFastAPIの Depends で共通化しており、エンドポイント個別には実装しない。新しい顧客向けAPIを追加する際は、必ず同じ依存を通すこと。
- セッション取得APIとログインAPIにはレート制限を設け、総当たり試行を抑止している。

2.3 DBアクセスの遮断

- 全テーブルでRLSを有効化し、ポリシーを一切作成していない。これによりAnon Key・Authenticated Keyからのアクセスは全て拒否される。
- Service Role KeyはRLSをbypassするため、Backendのみが通過できる。この鍵はBackendの環境変数にのみ存在し、フロントエンドには一切渡さない。
- フロントエンドはSupabaseクライアントを持たない。ライブラリを追加してブラウザから直接DBを触る実装は、この設計を根本から壊すため禁止。

2.4 秘密情報の一覧

変数名	保管場所	漏洩時の影響
SUPABASE_SERVICE_ROLE_KEY	Renderの環境変数	全データの読み書きが可能になる(最重要)
DATABASE_URL	Renderの環境変数	同上
JWT_SECRET	Renderの環境変数	スタッフになりすましたトークンを偽造できる
CRON_SECRET	Render / Vercelの環境変数	全卓を任意のタイミングで強制クリアできる

3. 仕様概観

3.1 主要機能

区分	機能
顧客	QRアクセスによるメニュー閲覧 / カート / 注文送信 / 注文履歴 / スタッフ呼出
スタッフ	ログイン / 卓ダッシュボード / 開栓 / 会計クリア / 提供済・Undo・取消 / 手入力(カスタム商品) / 品切れ設定
システム	日次バッチによる閉め忘れ卓の強制クリア

3.2 共通仕様のコア

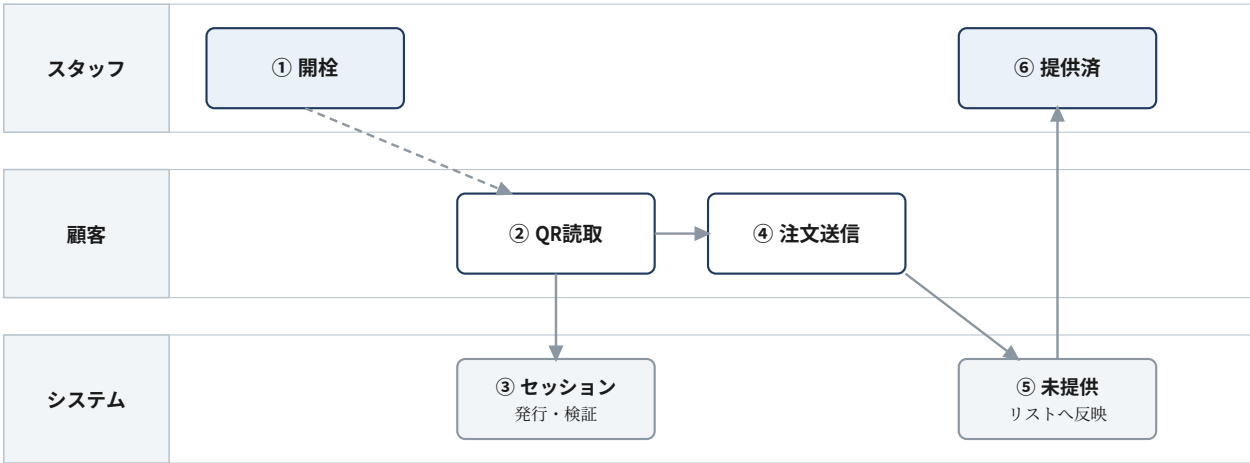
- 卓とセッション —
1卓につき同時にActiveな伝票は1件のみ。DBの部分UNIQUEインデックスで保証し、重複開栓は409を返す。
- 金額 — 税込価格をそのまま保持・表示する。店頭メニューが総額表示であり、税抜保持だと端数処理で記載額と1円ずれるため(税込1,000円は floor(税抜×1.1) で表現できない)。注文時点の価格・税率は明細にスナップショットされ、後からマスタを変えても過去伝票は変わらない。
- チャージ — システムでは扱わない。顧客画面の金額は参考値であり、正式な会計は既存レジ。
- 明細のステータス — pending / served / cancelled の3値。pending $\rightleftharpoons$ served と pending→cancelled のみ許可し、それ以外は409。サーバー側で必ず検証する。
- 画面更新 — WebSocketは使わず、SWRのポーリング(4秒 / 8秒)で反映する。
- エラー — 全APIが {"error":{"code","message"}} 形式で返す。フロントエンドはcodeで分岐し、messageをそのまま表示する。
- 権限 — スタッフ間の権限差はない。全スタッフが全操作を実行できる。

3.3 意図的に作っていないもの

以下は「実装漏れ」ではなく、実験導入のため意図的に対象外としている。追加を検討する際は [00_要件定義書] 2.2節の判断理由を確認すること。

対象外	代替
決済・会計処理	既存レジ
チャージ / 時間課金の計算	既存レジで加算
レシート・適格請求書の発行	既存レジ
スタッフの権限分離	全員同一権限
パスワードの自己リセット	管理担当者が手動で再設定
商品画像	テキストのみ
多言語対応	日本語のみ
売上集計・経理連携	なし(DBを直接参照)

4. 業務フロー(正常系)

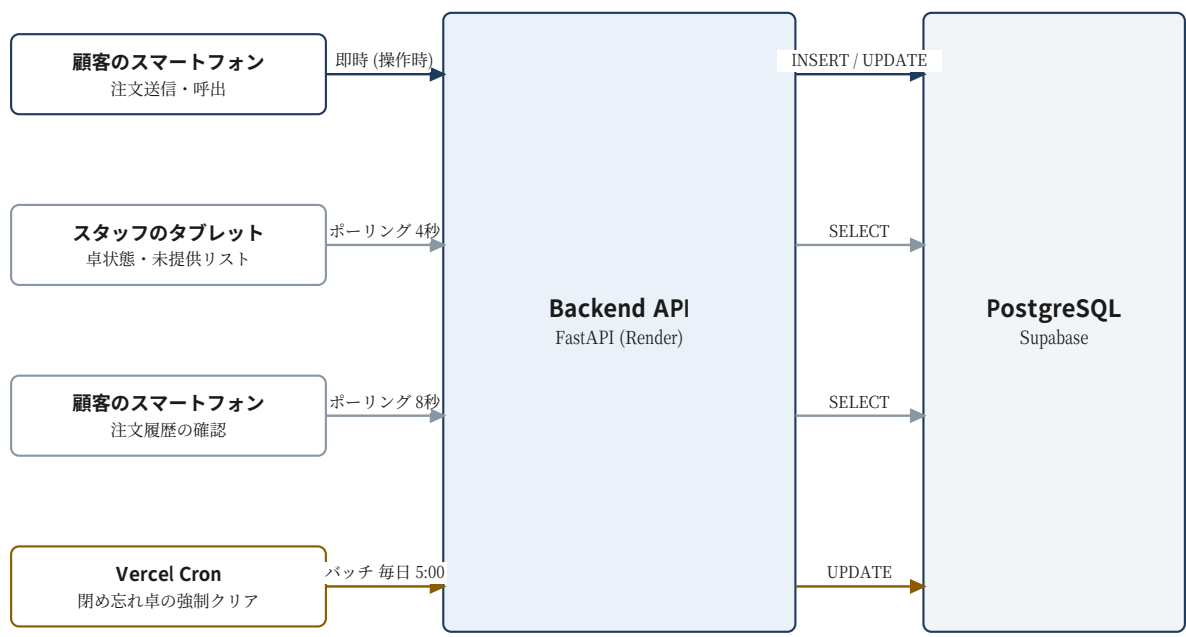


⑦ 客が会計を希望 → 「呼出」 実行 → ダッシュボードに呼出表示 → スタッフが既存レジで会計 → 「会計クリア」 で空席化
⑧ 閉め忘れた卓は日次バッチ(daily_clear)が翌朝に自動クリア(安全網)

図2 最も発生頻度が高い正常系フロー

#	操作	主なAPI	データの変化
1	スタッフが卓を開栓	PATCH /tables/{id}/activate	orders を1件INSERT (status=Active, session_token発行)
2	客がQRを読み取る	GET /tables/{id}/session	参照のみ
3	客がメニューを見る	GET /products	参照のみ(品切れ除外)
4	客が注文を送信	POST /orders/{id}/items	order_items を複数INSERT (status=pending)
5	スタッフ画面に反映	GET /staff/pending_items	参照のみ(4秒ポーリング)
6	スタッフが提供済にする	PATCH /order_items/{id}/status	status を served へUPDATE
7	客が会計を呼ぶ	POST /orders/{id}/call	is_calling を true へUPDATE
8	スタッフが会計クリア	PATCH /tables/{id}/clear	status=Deactivated, is_calling=false, checkout_staff_id を設定

5. データフロー



※ 顧客・スタッフの端末はDBへ直接アクセスしない。全ての読み書きがBackend APIを通る。

図3 データの経路とタイミング

経路	起点	タイミング	種別	主な操作
注文・呼出の書き込み	顧客端末	操作時(即時)	リアルタイム	INSERT / UPDATE
開栓・提供・クリア	スタッフ端末	操作時(即時)	リアルタイム	INSERT / UPDATE
卓状態・未提供リスト	スタッフ端末	4秒間隔	ポーリング	SELECT
注文履歴	顧客端末	8秒間隔	ポーリング	SELECT
閉め忘れ卓の強制クリア	Vercel Cron	毎日 5:00	バッチ	UPDATE

ポーリングを選んでいる理由

Supabase Realtimeをフロントエンドから直接購読すると、ブラウザに露出するAnon Keyでordersテーブルを購読できてしまい、session_tokenが漏洩し得る。これを避けるため、リアルタイム更新をあえて捨ててポーリングにしている。即時性が必要になった場合は、BackendがRealtimeを購読して認証済みWebSocketで中継する方式へ拡張する。

6. 主要インターフェース仕様

6.1 共通事項

項目	内容
プロトコル	HTTPS / REST
データ形式	JSON (UTF-8)
ベースパス	/api/v1
認証	Authorization: Bearer(スタッフ) / X-Session-Token(顧客) / X-Cron-Secret(バッチ)
エラー形式	{"error": {"code": "...", "message": "..."} }
文字コード	UTF-8固定
CORS	環境変数 ALLOWED_ORIGINS で許可オリジンを限定
APIドキュメント	FastAPI自動生成 (/docs) を正とする

6.2 エンドポイント一覧(概観)

区分	Method / Endpoint	認証	連携タイミング
顧客	GET /tables/{table_id}/session	なし(レート制限)	QR読取時
顧客	GET /products	なし	メニュー表示時
顧客	GET /orders/{order_id}/history	X-Session-Token	8秒ポーリング
顧客	POST /orders/{order_id}/items	X-Session-Token	注文送信時
顧客	POST /orders/{order_id}/call	X-Session-Token	呼出時
スタッフ	POST /staff/login	なし(レート制限)	ログイン時
スタッフ	POST /staff/logout	Bearer	ログアウト時
スタッフ	GET /tables	Bearer	4秒ポーリング
スタッフ	GET /staff/pending_items	Bearer	4秒ポーリング
スタッフ	GET /staff/products	Bearer	品切れ画面表示時
スタッフ	PATCH /staff/products/{id}/sold_out	Bearer	品切れ切替時
スタッフ	PATCH /tables/{table_id}/activate	Bearer	開栓時
スタッフ	PATCH /tables/{table_id}/clear	Bearer	会計クリア時
スタッフ	PATCH /order_items/{item_id}/status	Bearer	提供/Undo/取消時
スタッフ	POST /orders/{order_id}/custom_items	Bearer	手入力時
バッチ	POST /jobs/daily_clear	X-Cron-Secret	毎日 5:00

※ 型・桁数を含む詳細は [04_バックエンドAPI設計書] を参照。

6.3 外部システム連携

このシステムは既存Webサイト・予約システム・POSレジと一切連携していない。ドメイン・データベース・認証基盤がすべて独立しているため、本システムの変更が既存システムへ影響することはない(逆も同様)。

7. 設計上の不変条件(変更時の注意)

以下は、崩すとセキュリティ事故または業務事故に直結する。変更が必要になった場合は、影響範囲を検討したうえで設計書も併せて更新すること。

#	不変条件	崩れたときに起きること
1	フロントエンドにSupabaseクライアント・鍵を持たせない	ブラウザから全顧客の注文情報を直接読める
2	顧客系APIは必ず X-Session-Token を検証する(Depends経由)	卓IDの総当たりで他卓に注文・呼出・履歴閲覧ができる
3	RLSは全テーブル有効・ポリシー未作成のまま	Anon KeyでDBが読める
4	orders の部分UNIQUE (table_id) WHERE status='Active' を外さない	二重開栓で伝票が分裂し、会計金額が合わなくなる
5	ステータス遷移の検証をサーバー側で行う	クライアント改変で不正な遷移が通る
6	価格は明細にスナップショットする	価格改定時に過去の伝票金額が変わる
7	税込計算は1箇所(FEは shared/price.ts)に集約する	画面ごとに表示金額が食い違う
8	checkout_staff_id は日次バッチではNULLのまま残す	閉め忘れと正規クリアの区別がつかなくなる

8. リポジトリ構成

```
hihumi_manage/  
├── docs/    設計ドキュメント一式(00~08)  
├── db/  
│   ├── schema.sql  DDL(RLS有効化を含む)  
│   └── seed.sql    初期データ(7卓・商品マスタ)  
├── prototype/  操作プロトタイプ(単一HTML・モックデータ)  
├── frontend/   React + Vite  
└── backend/    FastAPI
```

探しているもの	見る場所
認証・セッション検証の実装	backend/app/deps.py
エラー形式の定義	backend/app/errors.py
APIの入出力スキーマ	backend/app/schemas.py
顧客向けAPI	backend/app/routers/customer.py
スタッフ向けAPI	backend/app/routers/staff.py
日次バッチ	backend/app/routers/jobs.py
卓の座標描画	frontend/src/features/staff/FloorMap.tsx
税込計算	frontend/src/shared/price.ts
APIクライアント(ヘッダ付与)	frontend/src/shared/api/client.ts
セッション保持	frontend/src/shared/session.ts

9. 運用タスクと監視ポイント

頻度	タスク	確認内容
日次(自動)	daily_clear バッチ	実行ログにエラーが出ていないか
日次(店舗)	開店前・閉店時のチェック	[07_利用ガイド] 4章のとおり
週次	エラーログの確認	403 / 409 が異常に多発していないか
月次	伝票データの確認	checkout_staff_id が NULL の伝票が常態化していないか
随時	商品マスタの更新	価格改定・季節メニューの反映
随時	スタッフアカウントの棚卸し	退職者のアカウントを is_active = false にする

9.1 異常の切り分け手順

#	確認
1	フロントエンドが開けるか → 開けなければ Vercel の状態を確認
2	API が応答するか(/docs にアクセス) → 応答しなければ Render の状態とログを確認
3	DBに接続できるか → Supabase の状態を確認
4	特定の操作だけ失敗するか → ブラウザの開発者ツールでエラーコードを確認し、[01_アーキテクチャ設計書] 8.1節のエラーコード一覧と突き合わせる
5	直近のデプロイが原因か → Vercel / Render のデプロイ履歴を確認し、必要ならロールバック

10. 拡張を検討する場合

やりたいこと	主な変更範囲	注意点
チャージを金額に含める	DB(課金テーブル or 定数) / 開栓時の人数入力 / 小計計算	スタッフの操作が1手増える。[00]の判断理由を再確認する
即時反映にする	BackendでRealtime購読 + WebSocket中継 / FEの購読処理	フロントエンドから直接購読させないこと(不変条件1)
スタッフ権限を分ける	staffにロール列追加 / Dependsで権限チェック	既存の全APIに権限判定を追加する必要がある
売上集計を出す	集計API or DB直接参照	cancelled明細の除外と、checkout_staff_id=NULL の扱いを決めること
卓の増減・レイアウト変更	tablesマスタのpos_x/pos_y更新 / QRコード再作成	コード修正は不要。データ更新のみで対応できる設計
多言語対応	FEの文言外部化 / 商品マスタの多言語列	商品名の翻訳運用をどうするか先に決める

11. 関連ドキュメント

番号	文書名	こういうときに読む
00	要件定義書	なぜこの仕様なのか、対象外の判断理由を知りたい
01	アーキテクチャ設計書	認証・RLS・レート制限・エラーコードの詳細
02	データベース設計書	テーブル定義・DDL・初期データ
03	フロントエンド設計書	画面仕様・ディレクトリ構成・状態管理

番号	文書名	こういうときに読む
04	バックエンドAPI設計書	API入出力の詳細・処理ロジック
05	開発・運用ガイド	環境構築・環境変数・デプロイ手順
06	テスト仕様書	受入テスト・セキュリティ確認項目
07	利用ガイド	店舗スタッフがどう使っているかを知りたい
08	Project Overview(本書)	全体像の把握・変更時の注意点

12. 変更履歴

版	主な変更内容
V1.0	新規作成。全体像・セキュリティ概要・仕様概観・正常系フロー・データフロー・主要I/F仕様に加え、設計上の不変条件、リポジトリの読みどころ、運用タスク、拡張時の変更範囲を収録。